

智能小车实训任务书

任务一、智能小车的连接及传输

一、任务描述

自第一台工业机器人诞生以来，机器人的发展已经遍及机械，电子，冶金，交通，宇航，国防等领域。近年来机器人的智能水平不断提高，并且迅速地改变着人们的生活方式。人们在不断探索，改造，认识自然的过程中，由此发展起来的智能小车引起了众多学者的广泛关注和极大的兴趣。

智能小车，也就是轮式机器人，最适合在那些人类无法工作的环境中工作，该技术可以应用于无人驾驶机动车，无人生产线，仓库，服务机器人，航空航天等领域。作为20世纪自动化领域的重大成就，机器人已经和人类社会的生产、生活密不可分。因此为了使智能小车工作在最佳状态，进一步研究及完善其速度和方向的控制是非常有必要的。

智能小车可以作为机器人的典型代表。它可以分为三大组成部分：传感器检测部分，执行部分，cpu。机器人要实现自动避障功能，还可以扩展循迹等功能，感知导引线 and 障碍物。可以实现小车自动识别路线，选择正确的行进路线，并检测到障碍物自动躲避。

二、任务目标

- 1、完成智能小车控制器（树莓派）的远程桌面连接
- 2、实现小车控制器（树莓派）与计算机的文件传输
- 3、掌握python的基础语法，能够完成简单编程

三、相关任务内容

1、智能小车组成

主控制板：选用树莓派4B Raspberry Pi

电源系统：智能小车采用了锂电池供电的方式，特点是工作噪音小、运行平稳、环保节能、价格便宜。由于各个系统会用到各种不同的电压，因此，除了电池之外，还需要准备降压模块。

减速电机：减速电机就是在直流电机基础上，增加一个高精密的减速器。一般直流电机的转速都是一分钟几千上万转，增加减速器可以降低电机转速，增加电机扭矩，使得可控性更强。选择减速电机时，电机的额定电压需要和选择的电池相匹配，常见的12V额定电压电机供电范围在11~16V之间，12V最佳。

2、VNC简介

VNC (Virtual Network Console)，即虚拟网络控制台，它是一款基于UNIX和Linux操作系统的优秀远程控制工具软件，由著名的AT&T的欧洲研究实验室开发，远程控制能力强大，高效实用，并且免费开源。

VNC基本上是由两部分组成：一部分是客户端的应用程序(vncviewer)；另外一部分是服务器端的应用程序(vncserver)。在任何安装了客户端的应用程序(vncviewer)的计算机都能十分方便地与安装了服务器端的应用程序(vncserver)的计算机相互连接。

3、VNC运行的工作流程

- 1) VNC客户端通过浏览器或VNC Viewer连接至VNC Server。
- 2) VNC Server传送一对话窗口至客户端，要求输入连接密码，以及存取的VNC Server显示装置。
- 3) 在客户端输入联机密码后，VNC Server验证客户端是否具有存取权限。

- 4) 若是客户端通过VNC Server的验证, 客户端即要求VNC Server显示桌面环境。
- 5) VNC Server通过X Protocol 要求X Server将画面显示控制权交由VNC Server负责。
- 6) VNC Server将来由 X Server的桌面环境利用VNC通信协议送至客户端, 并且允许客户端控制VNC Server的桌面环境及输入装置。

[如何使用VNC进行远程桌面控制 - 明志德道 - 博客园\(cnblogs.com\)](http://cnblogs.com/)

4、关于FileZilla

FileZilla 是一个跨平台的 FTP软件客户端, 具有图形用户界面(GUI), 并且支持Windows / MacOS / Linux 以及服务端四大平台, 功能强大, 操作简单直观, 是一款非常值得推荐的免费生产力工具。

5、FileZilla FTP 客户端特性

- 多年的维护, 更新迭代频率高, 软件速度快、稳定性高
- 界面简单易用, 几乎不需要学习, 支持多标签, 支持多种语言
- 支持 **FTP**、**FTPS**、**SFTP** 等常见的文件传输协议
- 自带站点管理, 方便管理多个网站项目, 同时支持远程搜索查找文件
- Windows端提供绿色免安装版下载, 打开就能用

四、任务要求

- 1、完成智能小车控制器(树莓派)的远程桌面连接
- 2、实现小车控制器(树莓派)与计算机的文件传输
- 3、掌握python的基础语法, 能够完成简单编程

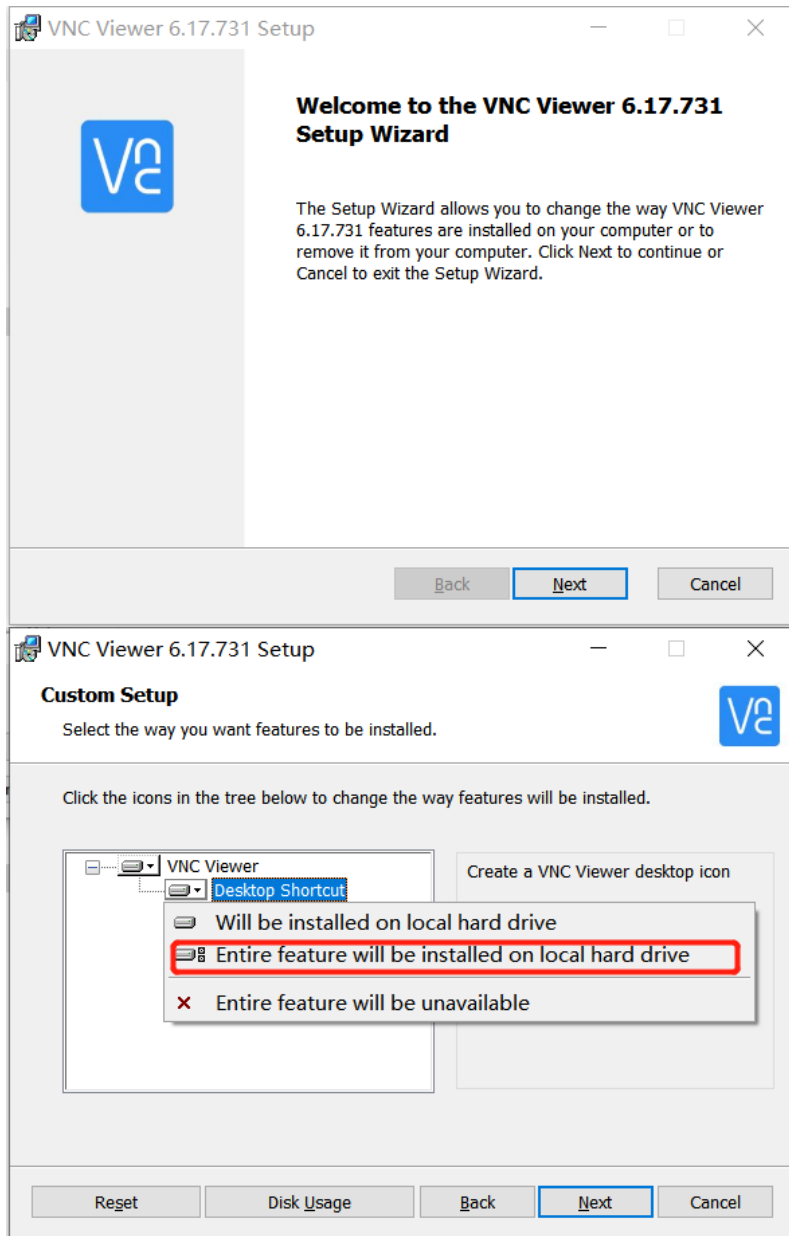
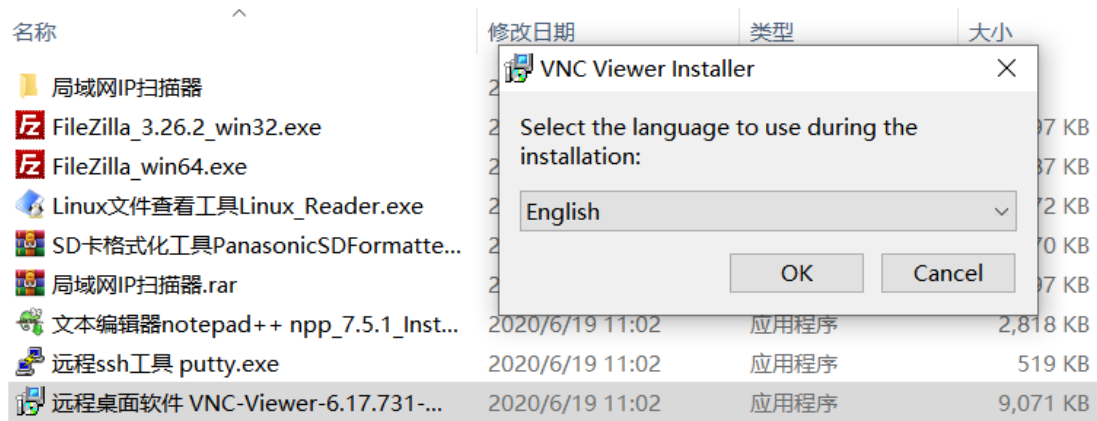
五、任务实施

1、安装辅助软件

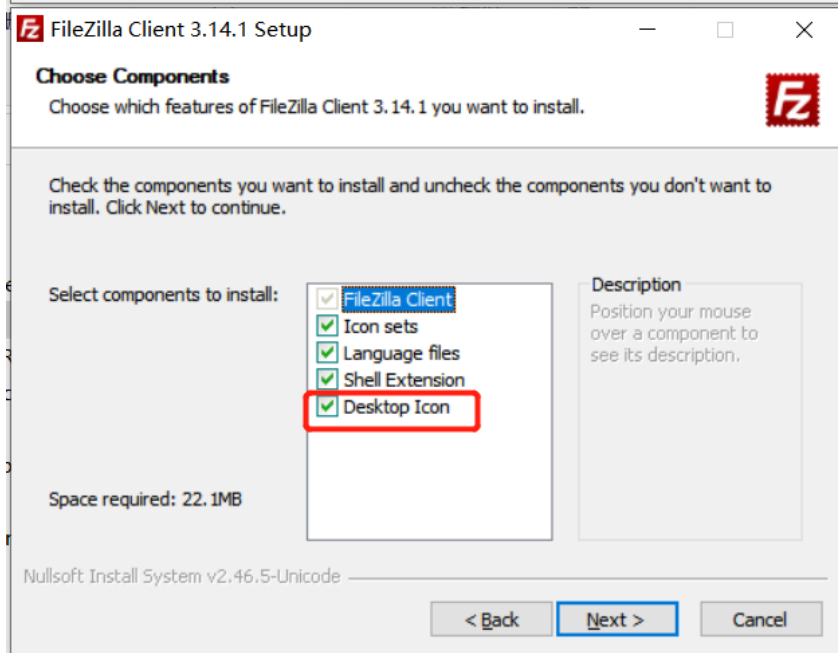
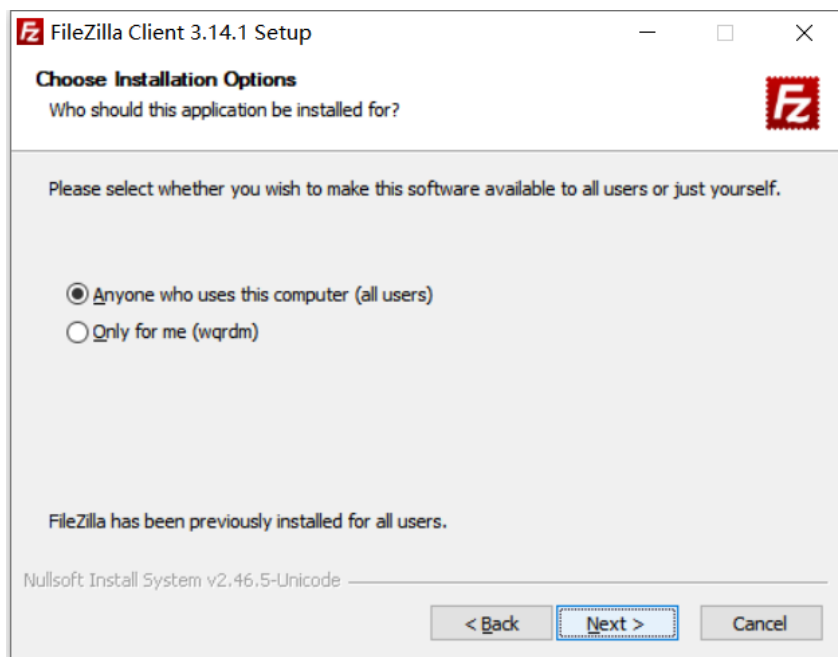
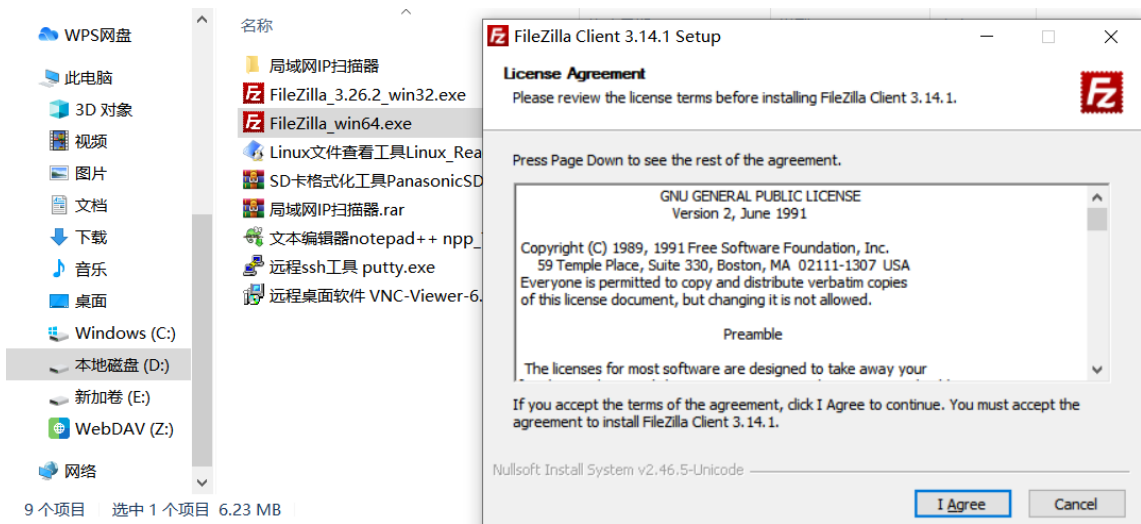
将安装包文件下载或拷贝至自己的计算机中

名称	修改日期	类型	大小
局域网IP扫描器	2020/6/1 14:51	文件夹	
 FileZilla_3.26.2_win32.exe	2020/6/19 11:02	应用程序	6,697 KB
 FileZilla_win64.exe	2020/6/19 11:02	应用程序	6,387 KB
 Linux文件查看工具Linux_Reader.exe	2020/6/19 11:02	应用程序	20,472 KB
 SD卡格式化工具PanasonicSDFormatte...	2020/6/19 11:02	WinRAR 压缩文件	370 KB
 局域网IP扫描器.rar	2020/6/19 11:02	WinRAR 压缩文件	7,197 KB
 文本编辑器notepad++ npp_7.5.1_Inst...	2020/6/19 11:02	应用程序	2,818 KB
 远程ssh工具 putty.exe	2020/6/19 11:02	应用程序	519 KB
 远程桌面软件 VNC-Viewer-6.17.731-...	2020/6/19 11:02	应用程序	9,071 KB

双击远程桌面软件VNC-Viewer的安装包

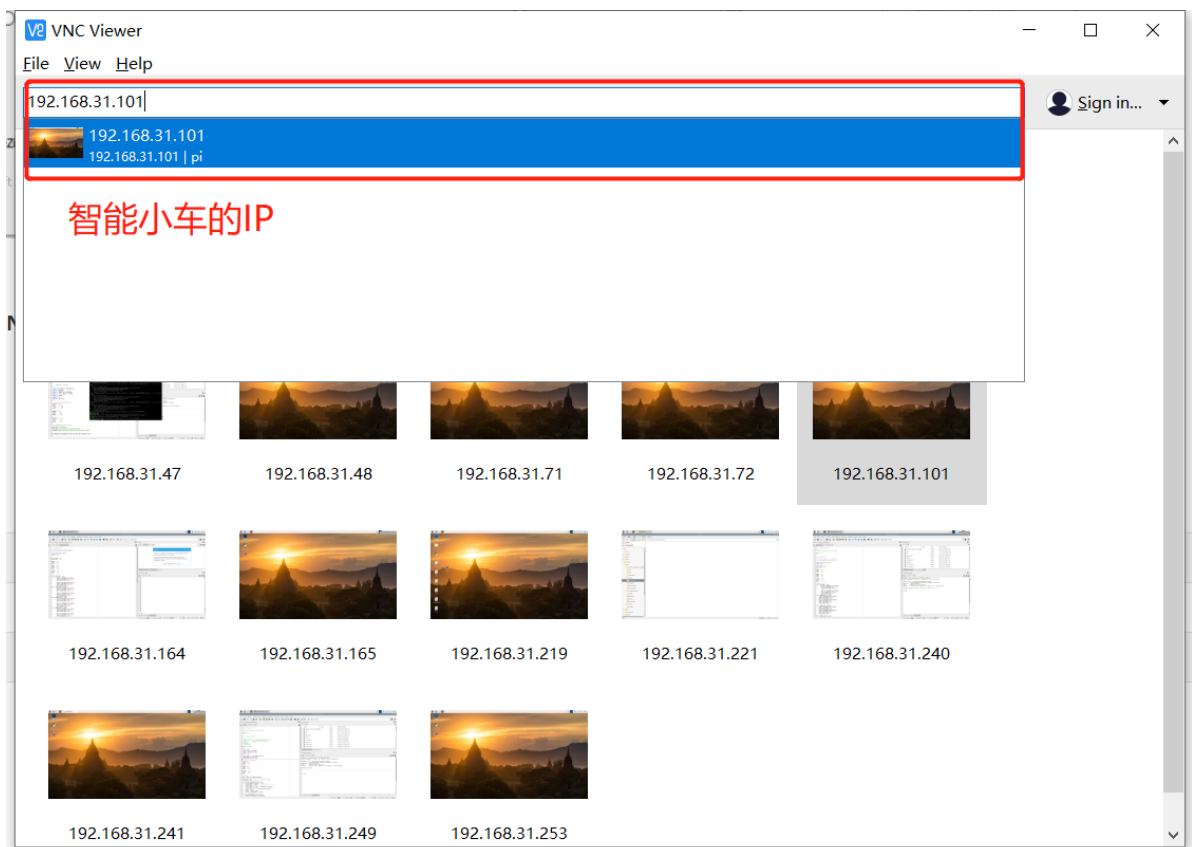


双击FileZilla的安装包

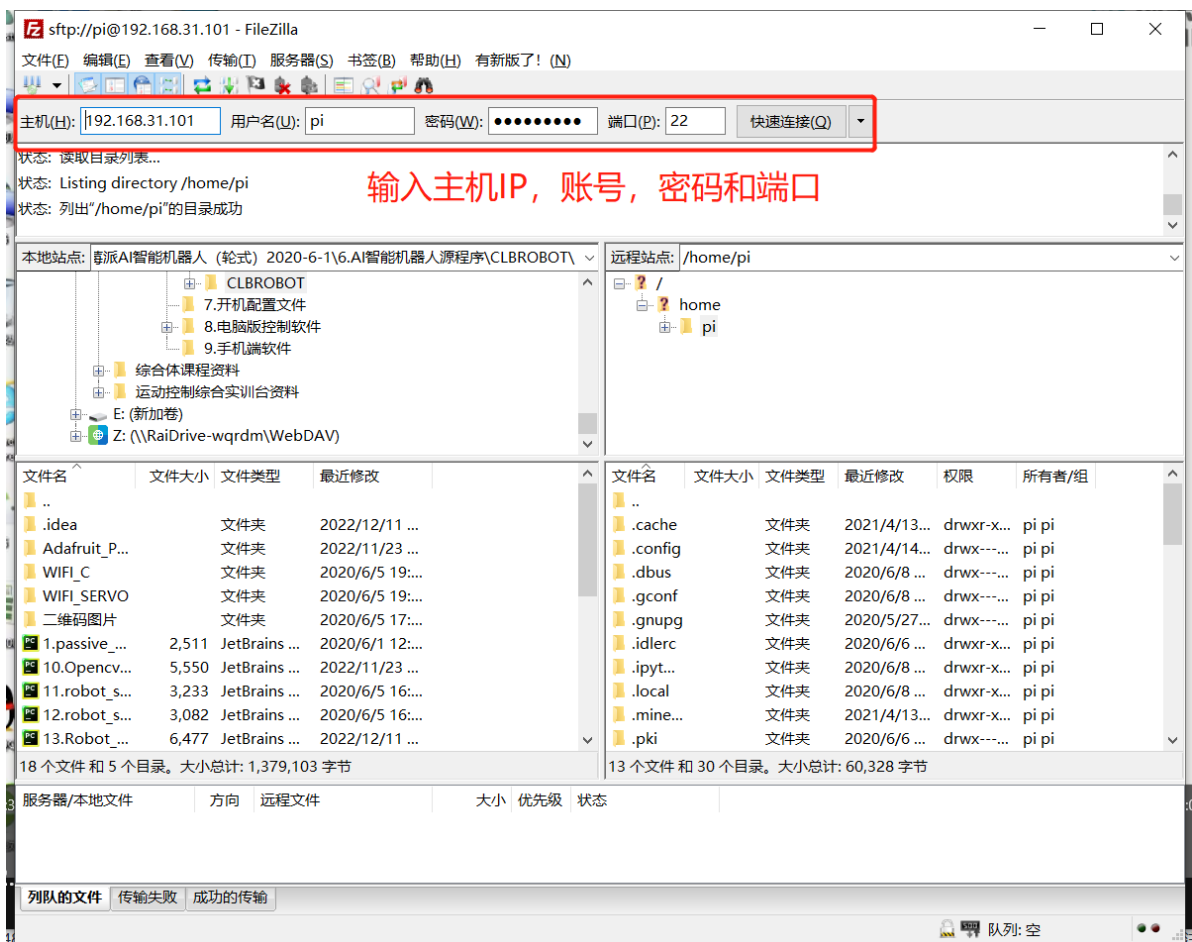


2、VNC-Viewer及FileZilla的使用

打开VNC Viewer，输入自己组智能小车的IP，登入账号：pi，密码：raspberry



打开FileZilla，输入自己组智能小车的IP，登入账号：pi，密码：raspberrypi，端口：22。



连接成功后，可将左边计算机文件拖入至右边小车树莓派中。

六、任务小结

任务二、智能小车基本运动的实现

一、任务描述

前进、后退、左转、右转、停止动作是智能小车的基本运动，掌握小车的运动方式并调节参数改变小车的运动形式是本项目的重要内容。

二、任务目标

- 1、实现小车的前进、后退、左转、右转、停止动作，并能调整运动参数
- 2、小车运动由功能键进行启动及关闭
- 3、实现小车固定轨迹运动（三角形、矩形、圆形等），计算圆形轨迹半径的表达式，并简单验证

三、相关任务内容

PWM控制技术

PWM控制技术以其控制简单，灵活和动态响应好的优点而成为电力电子技术最广泛应用的控制方式，也是人们研究的热点。由于当今科学技术的发展已经没有了学科之间的界限，结合现代控制理论思想或实现无谐振波开关技术将会成为PWM控制技术发展的主要方向之一。

其根据相应载荷的变化来调制晶体管基极或MOS管栅极的偏置，来实现晶体管或MOS管导通时间的改变，从而实现开关稳压电源输出的改变。这种方式能使电源的输出电压在工作条件变化时保持恒定，是利用微处理器的数字信号对模拟电路进行控制的一种非常有效的技术。

资源使用	资源类型	备注
直流减速电机方向控制*2	AIN1: 22, AIN2: 27; BIN1: 25, BIN2: 24 (GPIO)	需要 4 个 IO 口
直流减速电机速度控制*2	18, 23 (PWMA, PWMB)	需要 2 个 PWM 外设
按键	19 (GPIO)	启动按键 (高电平有效)
避障传感器	16, 12 (GPIO)	障碍物传感 (高电平点亮)

直流电机的转动

PIN正 引脚输出高电平，PIN负 引脚输出低电平，电机即可正转。

PIN正 引脚输出低电平，PIN负 引脚输出高电平，电机即可反转。

直流电机的转速

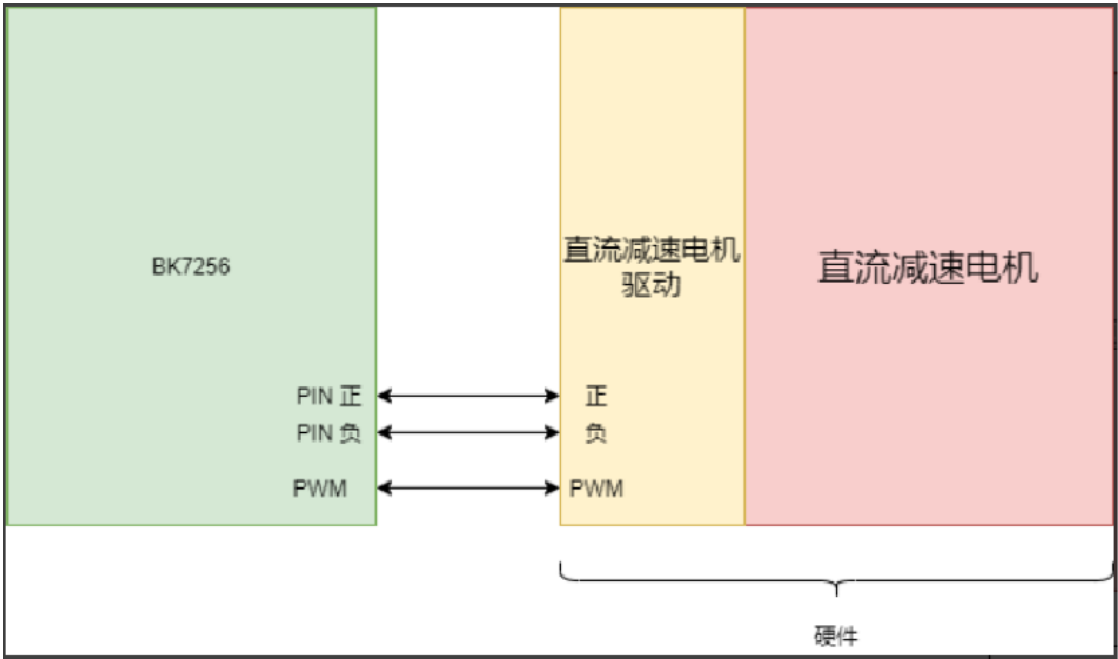
当 PWM 输出最大占空比的时候，电机全速转动。

注意事项

频繁切换电机的转向，可能会烧坏直流减速电机驱动器。
直流减速电机是相对安装的。对于小车来说，一个电机正转，一个电机反转，才是正转。

小车转向

左边电机前进，右边电机后退，小车右转。
左边电机后退，右边电机前进，小车左转。



直流电机	旋转方式	AIN1	AIN2	BIN1	BIN2
L	正转	高	低	/	/
	反转	低	高	/	/
	停止	低	低	/	/
R	正转	/	/	高	低
	反转	/	/	低	高
	停止	/	/	低	低

```
#无视警告，开启引脚
GPIO.setwarnings(False)
# 设置编码规范
GPIO.setmode(GPIO.BCM)
```

这里我们必须设置忽略GPIO警告，不然无法进行下面操作（可以自行尝试不输入这句代码，看看会怎么样）

引脚有两种模式：

- 1、输出模式：向引脚发送高低电平信号
 - 2、输入模式：读取引脚状态（高低电平）
- 因为要控制小车，所以在这里我们把所有引脚设置为输出

```
#设置引脚为输出
GPIO.setup([AIN1, AIN2, BIN1, BIN2, PWMA, PWMB], GPIO.OUT)
```

脉宽调制（PWM）基本原理：控制方式就是对逆变电路开关器件的通断进行控制，使输出端得到一系列幅值相等的脉冲，用这些脉冲来代替正弦波或所需要的波形。也就是在输出波形的半个周期中产生多个脉冲，使各脉冲的等值电压为正弦波形，所获得的输出平滑且低次谐波少。按一定的规则对各脉冲的宽度进行调制，即可改变逆变电路输出电压的大小，也可改变输出频率。

我们为了能够控制小车的速度，所以要采用PWM控制方法

PWM可以简单理解为，通过快速的高低电平的切换，达到控制电机的有效电压的效果，进而控制小车的速度

```
#对使能引脚开启pwm控制，频率为100Hz
L_Motor= GPIO.PWM(PWMA,100)
# 启动pwm
L_Motor.start(0)

R_Motor = GPIO.PWM(PWMB,100)
R_Motor.start(0)
```

占空比是指在一个脉冲循环内，通电时间相对于总时间所占的比例。占空比(Duty Ratio)在电信领域中有如下含义：例如：脉冲宽度 $1\mu s$ ，信号周期 $4\mu s$ 的脉冲序列占空比为0.25。

基本含义：

在一段连续工作时间内脉冲占用的时间与总时间的比值。

引申义：

在周期型的现象中，某种现象发生后持续的时间与总时间的比。

例如，在俗语中有句话：「三天打渔，两天晒网」，是说一共三天，有一天是打渔，两天是晒渔网。打渔的占空比为 $1/3$ 。

理论定义

占空比是指电路被接通的时间占整个电路工作周期的百分比。比如说，一个电路在它一个工作周期中有一半时间被接通了，那么它的占空比就是50%。如果加在该工作元件上的信号电压为5V，则实际的工作电压平均值或电压有效值就是2.5V。假设该元件为一个电子阀门，当电路全部接通时，阀门全开；当占空比为50%时，阀门状态为半开。同理，当占空比设置为20%时，阀门的开度显然应该为20%。这样,这个阀门就可以在0%(全闭)到100%（全开）的范围内任意调节。

```
def t_up(speed,t_time):
    #更改占空比
    L_Motor.ChangeDutyCycle(speed)
    GPIO.output(AIN2,False)#AIN2
    GPIO.output(AIN1,True) #AIN1

    R_Motor.ChangeDutyCycle(speed)
    GPIO.output(BIN2,False)#BIN2
    GPIO.output(BIN1,True) #BIN1
    # 持续t_time秒
    time.sleep(t_time)
```

sleep() 方法暂停给定秒数后执行程序。该参数可以是一个浮点数来表示一个更精确的睡眠时间。

time.sleep(t)

t -- 这是要暂停执行的秒数。

小车行进档位

档位	零档	一档	二档	三档	四档	五档
PWM 占空比	0	20%	40%	60%	80%	100%

使用PWM控制，让小车速度在10S内逐渐加快

```
#让小车每秒钟逐渐增加速度
for i in range(11):
    PWMA.ChangeDutyCycle(10 * i)
    PWMB.ChangeDutyCycle(10 * i)
    time.sleep(1)
```

通过功能按钮，来启动小车运动

```
#定义引脚
BtnPin = 19
Gpin = 5
Rpin = 6
GPIO.setup(Gpin, GPIO.OUT)      # 设置绿色Led引脚模式输出
GPIO.setup(Rpin, GPIO.OUT)      # 设置红色Led引脚模式输出
GPIO.setup(BtnPin, GPIO.IN, pull_up_down=GPIO.PUD_UP)  # 设置输入BtnPin模式，拉高至高电平(3.3V)
#定义启动函数
def keysacn():
    val = GPIO.input(BtnPin)
    while GPIO.input(BtnPin) == False:
        val = GPIO.input(BtnPin)
    while GPIO.input(BtnPin) == True:
        time.sleep(0.01)
        val = GPIO.input(BtnPin)
        if val == True:
            GPIO.output(Rpin,1)
            while GPIO.input(BtnPin) == False:
                GPIO.output(Rpin,0)
        else:
            GPIO.output(Rpin,0)
```

Python异常处理 - Try, Except

Python有许多[内置的异常](#)，当其中的某些错误出现时，它们会强制您的程序输出错误。

当发生这些异常时，它将导致当前进程停止并将其传递给调用进程，直到被处理为止。如果不处理，我们的程序将崩溃。

在Python中，可以使用try语句处理异常。可能引发异常的关键操作放在try子句中，并且将处理异常的代码编写在except子句中。

```
try:
    <代码块1>
except 异常类型1 : # KeyboardInterrupt为异常类型，当按下Ctrl+C时，执行代码块2
    <代码块2>
```

四、任务要求

- 1、能够流畅的实现各运动要求
- 2、可由功能键启动小车的运动
- 3、实现小车固定轨迹运动（三角形、矩形、圆形等）

五、任务实施

1、打开PyCharm软件，新建一个项目文件，可命名为basic motion。

完成以下的完整运动程序

```
# 完整程序
#导入库文件
import RPi.GPIO as GPIO
import time
#定义引脚
PWMA = 18
AIN1 = 22
AIN2 = 27
...

#声明定义相关函数
def t_up(speed,t_time):
    L_Motor.ChangeDutyCycle(speed)
    GPIO.output(AIN2,False)    #AIN2
    GPIO.output(AIN1,True)     #AIN1
    R_Motor.ChangeDutyCycle(speed)
    GPIO.output(BIN2,False)    #BIN2
    GPIO.output(BIN1,True)     #BIN1
    time.sleep(t_time)
#t_down、t_left、t_right、t_stop结合t_up函数和上述相关知识自行完成
#设备初始化，开启引脚并设置模式
GPIO.setwarnings(False)
GPIO.setmode(GPIO.BCM)
GPIO.setup(AIN2,GPIO.OUT)
GPIO.setup(AIN1,GPIO.OUT)
GPIO.setup(PWMA,GPIO.OUT)
...
#对使能引脚开启pwm控制，并启动pwm
L_Motor= GPIO.PWM(PWMA,100)
L_Motor.start(0)
...
#调用函数，运行主程序
keysacn()
try:
    while True:
        t_up(50,3)
        t_down(50,3)
        t_left(50,3)
        t_right(50,3)
        t_stop(3)
except KeyboardInterrupt: # 当按下Ctrl+C时，退出
    GPIO.cleanup()
```

2、将程序文件传输至树莓派中。

3、通过远程桌面运行该程序文件，按下功能按钮，实现小车运动。

六、任务小结

七、任务拓展

监听、操作鼠标、键盘是实现自动化的捷径，比如我实现自动化签到用到了模拟键盘操作。

pynput是监听、操控鼠标和键盘的跨平台第三方python库。

你可以通过pip install pynput来安装。安装时会自动下载依赖库。

相关函数的使用可查看[pynput · PyPI](#)以及[python监听、操作键盘鼠标库pynput详细教程 - 风吹云动 - 博客园\(cnblogs.com\)](#)

通过调用pynput第三方库，实现键盘遥控小车运动。

通过调用pynput第三方库，可切换小车运动模式（固定轨迹切换，红外避障运动等不同模式）。

任务三、红外避障运动

一、任务描述

自主识别前方障碍物，并根据障碍物位置信息实现避障。

二、任务目标

- 1、小车能够实现避障功能
- 2、在硬件条件的允许下，优化小车的避障功能

三、相关任务内容

红外避障传感器引脚为16和12。将两个引脚设置为输入模式。

```
SensorRight = 16
SensorLeft = 12
GPIO.setup(SensorRight,GPIO.IN)
GPIO.setup(SensorLeft,GPIO.IN)
```

根据前端的左右红外传感器信号，通过条件语句，判断前方的障碍物位置。根据不同的条件结果，调用合适的运动函数，实现避障。

```
try:
    while True:
        SR_2 = GPIO.input(SensorRight)
        SL_2 = GPIO.input(SensorLeft)
        if （传感器信息作为判断条件，左右两端无障碍）:
            print("t_up")
        elif （传感器信息作为判断条件，左端无障碍）:
            print("Left")
```

```

        elif （传感器信息作为判断条件，右端无障碍）：
            print( "Right")
        else:
            print( "Adjust the position")

except KeyboardInterrupt:  # 当按下Ctrl+C时，退出。
    GPIO.cleanup()

```

四、任务要求

- 1、小车能够实现避障功能
- 2、在硬件条件的允许下，优化小车的避障功能

五、任务实施

- 1、打开PyCharm软件，新建一个项目文件，可命名为infrad_avoid。

根据上述相关内容，完成完整的避障运动程序，以下为主函数代码。

```

if __name__ == '__main__':
    setup()#运动初始化设置，自行添加该函数
    keysacn()
    L_Motor= GPIO.PWM(PWMA,100)
    L_Motor.start(0)
    R_Motor = GPIO.PWM(PWMB,100)
    R_Motor.start(0)
    try:
        while True:
            SR_2 = GPIO.input(SensorRight)
            SL_2 = GPIO.input(SensorLeft)
            if （传感器信息作为判断条件，左右两端无障碍）：
                print("t_up")

            elif （传感器信息作为判断条件，左端无障碍）：
                ...

            elif （传感器信息作为判断条件，右端无障碍）：
                ...

            else:
                ...

    except KeyboardInterrupt:  # 当按下Ctrl+C时，退出。
        GPIO.cleanup()

```

- 2、将程序文件传输至树莓派中。
- 3、通过远程桌面运行该程序文件，按下功能按钮，实现小车运动。

六、任务小结

任务四、OpenCV视觉循迹运动

一、任务描述

根据道路上的车道线，通过智能小车上的摄像头进行识别，使小车沿着车道线运动。

二、任务目标

- 1、小车能实现循迹功能
- 2、对于车道线的不同情况做成合理的运动判断

三、相关任务内容

读取图像

```
import cv2
img = cv2.imread("Resources/lena.png")
cv2.namedWindow('Lena Soderberg',cv2.WINDOW_NORMAL)
cv2.imshow("Lena Soderberg",img)
cv2.waitKey(0)      #等待键盘输入，为毫秒级
cv2.destroyWindow("Lena Soderberg")
cv2.destroyAllWindows()  #可以轻易删除任何我们建立的窗口，括号内输入想删除的窗口名
```

视频文件处理

cv2.VideoCapture()：0为默认计算机默认摄像头，1可以更换来源；

cv2.VideoCapture('文件路径')

```
import cv2
cap = cv2.VideoCapture(0)
cap.set(3,640)#视频流宽度
cap.set(4,480)#视频流高度
cap.set(10,100)#视频流亮度
while True:
    #capture frame-by-frame
    ret , frame = cap.read()
    gray = cv2.cvtColor(frame , cv2.COLOR_BGR2GRAY)
    cv2.imshow('frame',gray)
    if cv2.waitKey(1) &0xFF ==ord('q'):  #按q键退出
        break
cap.release()#释放摄像头设备资源
cv2.destroyAllWindows()
```

灰度化

灰度数字图像是每个像素只有一个采样颜色的图像。这类图像通常显示为从最暗黑色到最亮的白色的灰度，尽管理论上这个采样可以任何颜色的不同深浅，甚至可以是不同亮度上的不同颜色。灰度图像与黑白图像不同，在计算机图像领域中黑白图像只有黑白两种颜色，灰度图像在黑色与白色之间还有许多级的颜色深度。

```
#图像转化为灰度图
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
#高斯模糊
imgBlur=cv2.GaussianBlur(gray,(7,7),0)
#边界检测
imgCanny = cv2.Canny(img,150,200)
#图像膨胀
import numpy as np
kernel = np.ones((5,5),np.uint8)
dilation = cv2.dilate(Canny,kernel,iterations=1)
#iterations迭代次数
#图像腐蚀
erosion = cv2.erode(dilation,kernel,iterations=1)
#图像二值化
ret,thresh1 = cv2.threshold(blur,60,255,cv2.THRESH_BINARY_INV)
#blur:输入图像
#60: 是图像的分界线
#255: 是超过图像的后变成的值
#cv2.THRESH_BINARY_INV: 为算法参数
```

轮廓查找

```
contours,hierarchy = cv2.findContours(mask.copy(), 1, cv2.CHAIN_APPROX_NONE)
#查找物体轮廓（白色那段），只接受二值图（黑白）
#mask.copy: 是寻找轮廓的图像
#cv2.CHAIN_APPROX_NONE: 存储所有的轮廓点，相邻的两个点的像素位置差不超过1，即max(abs(x1-x2), abs(y2-y1)) == 1
```

面积筛选及各中心点计算

```
for c in contours:
    area = cv2.contourArea(c)
    print(area)
    if area>20000:
        M = cv2.moments(c)
        cx = int(M['m10'] / M['m00'])
        cy = int(M['m01'] / M['m00'])
```

中心点计算

```
if len(contours) > 0: #判断是否有物体的轮廓
    #len(contours)物体的轮廓存储点数（只存储可以用直线描述轮廓的点的个数）
    c = max(contours, key=cv2.contourArea) #得到区域的面积中的选出最大的区域
    M = cv2.moments(c) #图像矩通常用来描述 分割 后的图像对象。可以通过图像的矩来获得图像的部分性质，包括面积(或总体亮度)，以及有关 几何中心 和 方向 的信息 。矩可以理解为图像的各类几何特征
    cx = int(M['m10']/M['m00'])#计算M中心X轴
    cy = int(M['m01']/M['m00'])#计算M中心Y轴
    print cx #打印中心X轴
```

舵机的控制

```
import Adafruit_PCA9685

pwm = Adafruit_PCA9685.PCA9685()

channelA=5 # 底座舵机通道
channelB=4 # 顶部舵机通道
# 辅助功能，使设置舵机角度更简单
def set_servo_angle(channel,angle):
    angle=4096*((angle*11)+500)/20000+0.5
    pwm.set_pwm(channel,0,int(angle))
# 频率设置为50hz，适用于舵机系统。
pwm.set_pwm_freq(50)
set_servo_angle(channelA,90) # 底座舵机 90
set_servo_angle(channelB,145) # 顶部舵机 145
```

四、任务要求

- 1、小车能实现循迹功能
- 2、对于车道线的不同情况做成合理的运动判断

五、任务实施

- 1、打开PyCharm软件，新建一个项目文件。

根据上述相关内容，完成完整的循迹运动程序，以下为部分代码。

```
#导入相关库
import cv2
import Adafruit_PCA9685
import RPi.GPIO as GPIO
...
#打开摄像头，同时可设置视频流参数
...
#设备初始化设置
...
#完成顶部和底座舵机的角度控制
pwm = Adafruit_PCA9685.PCA9685()
...
...
#等待舵机完成运动
time.sleep(0.5)
#调用启动按钮函数
...

while 1:
    ret, frame = video_capture.read()
    crop_img = frame[60:120, 0:160]
    gray = cv2.cvtColor(crop_img, cv2.COLOR_BGR2GRAY)
    blur = cv2.GaussianBlur(gray, (5,5), 0)
    ret, thresh1 = cv2.threshold(blur, 60, 255, cv2.THRESH_BINARY_INV)
    mask = cv2.erode(thresh1, None, iterations=2)
    mask = cv2.dilate(mask, None, iterations=2)
```

```

contours,hierarchy = cv2.findContours(mask.copy(), 1, cv2.CHAIN_APPROX_NONE)
if len(contours) > 0:
    c = max(contours, key=cv2.contourArea)
    M = cv2.moments(c)
    cx = int(M['m10']/M['m00'])
    cy = int(M['m01']/M['m00'])
    cv2.line(crop_img,(cx,0),(cx,720),(255,0,0),1)
    cv2.line(crop_img,(0,cy),(1280,cy),(255,0,0),1)
    cv2.drawContours(crop_img, contours, -1, (0,255,0), 1)
    if cx >= 120:
        t_right(50,0)
        print( "Turn Left!")
    if cx < 120 and cx > 50:
        t_up(50,0)
        print ("On Track!")
    if cx <= 50:
        t_left(50,0)
        print ("Turn Right")
else:
    print ("I don't see the line" )
#Display the resulting frame
cv2.imshow('frame',crop_img)
if cv2.waitKey(1) & 0xFF == ord('q'):
    L_Motor.stop()
    R_Motor.stop()
    GPIO.cleanup()
    break

```

- 2、将程序文件传输至树莓派中。
- 3、搭建循迹场地
- 4、通过远程桌面运行该程序文件，按下功能按钮，实现小车运动。

六、任务小结

七、任务拓展

将单车道线循迹扩展为双车道线循迹项目

其流程大致如下：

车道线检测

图像二值化，提取车道线

提取车道线的内侧点：在图像的特定行，从中间向两侧检索，检测到0像素点及为车道线点。图像左半边检测到的点即为左车道线点，右半边检测到的点即为右车道线点。（为防止因各种原因车道线部分缺失，我们选择图像4个行提取4组车道线内侧点）

运动控制

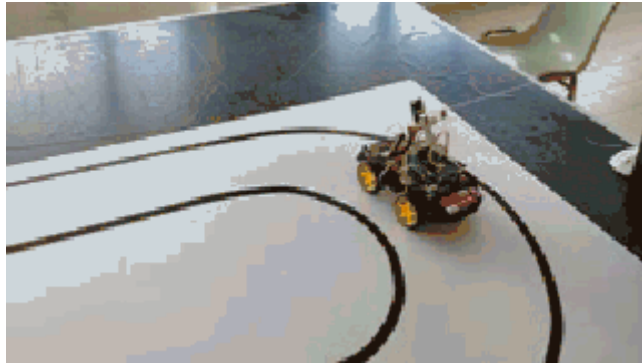
运动决策

如果两侧车道线都能检测到，则直行

如果只能检测到一侧车道线，则有三种情况：

该侧车道线靠近图像边缘，则还可以继续直行

该侧车道线靠近图像中央，则急需转弯，原地旋转
该侧车道线位置适中，则缓慢转弯，在前进中转弯
如果两侧车道线都检测不到，则维持之前的动作
电机控制：根据决策结果，控制小车电机输出



任务五、颜色球体追踪运动

一、任务描述

通过小车上的摄像头，对前方的颜色球体进行识别。根据两者相对位置，调整小车运动。

二、任务目标

- 1、小车能实现追踪功能
- 2、对于球体位置的不同情况做成合理的运动判断

三、相关任务内容

滑动条创建

```
import cv2

def nothing(x):
    pass
cv2.namedWindow('image')

cv2.createTrackbar('R','image',0,255,nothing)
cv2.createTrackbar('G','image',0,255,nothing)
cv2.createTrackbar('B','image',0,255,nothing)

r = cv2.getTrackbarPos('R','image')
g = cv2.getTrackbarPos('G','image')
b = cv2.getTrackbarPos('B','image')
cv2.waitKey(0)
```

HSV转换，应用阈值

```
#转换到HSV
hsv = cv2.cvtColor(frame,cv2.COLOR_BGR2HSV)
hue,sat,val = cv2.split(hsv)#三通道赋值
# 从滑动条获取信息
hmn = cv2.getTrackbarPos('hmin','HueComp')
hmx = cv2.getTrackbarPos('hmax','HueComp')

# 应用阈值
hthresh = cv2.inRange(hue,hmn,hmx)
# h s和v
tracking = cv2.bitwise_and(hthresh,cv2.bitwise_and(sthresh,vthresh))
```

图片过滤处理

```
#一些morphological过滤
dilation = cv2.dilate(tracking,kernel,iterations = 1)
closing = cv2.morphologyEx(dilation, cv2.MORPH_CLOSE, kernel)
closing = cv2.GaussianBlur(closing,(5,5),0)
'''

cv2.morphologyEx(src, op, kernel) 进行各类形态学的变化
参数说明：
src传入的图片
op进行变化的方式
kernel表示方框的大小

op = cv2.MORPH_OPEN 进行开运算，指的是先进行腐蚀操作，再进行膨胀操作
op = cv2.MORPH_CLOSE 进行闭运算， 指的是先进行膨胀操作，再进行腐蚀操作
开运算：表示的是先进行腐蚀，再进行膨胀操作
闭运算：表示先进行膨胀操作，再进行腐蚀操作
'''
```

检测圆形物体

```
#使用HoughCircles检测圆
circles = cv2.HoughCircles(closing,cv2.HOUGH_GRADIENT,2,120,
param1=120,param2=50,minRadius=10,maxRadius=0)
'''

HoughCircles(image, method, dp, minDist, circles=None, param1=None, param2=None,
minRadius=None, maxRadius=None)
参数说明：
'''
```

参数	含义
image	原始图像
method	目前只支持cv2.HOUGH_GRADIENT
dp	图像解析的反向比例。1为原始大小，2为原始大小的一半
minDist	圆心之间的最小距离。过小会增加圆的误判，过大会丢失存在的圆
param1	Canny检测器的高阈值
param2	检测阶段圆心的累加器阈值。越小的话，会增加不存在的圆；越大的话，则检测到的圆就更加接近完美的圆形
minRadius	检测的最小圆的半径
maxRadius	检测的最大圆的半径

```
#画圆圈
if circles is not None:
    x, y, r = circles[0][0]
    x_p = int(round(x))
    print(x_p)
    for i in circles[0,:]:
        # 如果球很远, 用绿色画出来
        if int(round(i[2])) < 30:
            cv2.circle(frame,(int(round(i[0])),int(round(i[1]))),int(round(i[2])),
(0,255,0),5)
            cv2.circle(frame,(int(round(i[0])),int(round(i[1]))),2,(0,255,0),10)
        # 或者用红色画
        elif int(round(i[2])) > 35:
            cv2.circle(frame,(int(round(i[0])),int(round(i[1]))),int(round(i[2])),
(0,0,255),5)
            cv2.circle(frame,(int(round(i[0])),int(round(i[1]))),2,(0,0,255),10)
```

四、任务要求

- 1、小车能实现追踪功能
- 2、对于球体位置的不同情况做成合理的运动判断

五、任务实施

- 1、根据下列开发流程，完成颜色球体识别运动程序



六、任务小结

任务六、二维码识别运动

一、任务描述

通过小车上的摄像头，对前方的二维码进行识别。根据二维码信息，调整小车运动。

二、任务目标

- 1、小车能实现二维码识别
- 2、对于二维码位置及二维码信息的不同情况做成合理的运动判断

三、相关任务内容

第三方库安装：

生成二维码：pip install qrcode。

读取二维码：pip install pyzbar。

```
#简单生成二维码
import qrcode
img = qrcode.make('文本')
img.save('123.jpg')
```

```
#生成二维码
import qrcode
qr =
qrcode.QRCode(version=5,error_correction=qrcode.constants.ERROR_CORRECT_H,box_size=8,border=4)
qr.add_data('你好')
qr.make(fit=True)
img=qr.make_image()
img.show()
img.save('123.jpg')
'''
```

参数说明：

version: 控制二维码的大小，取值范围从1到40。取最小值1时，二维码大小为21*21。取值为 **None** （默认）或者使用**fit=true**参数（默认）时，二维码会自动调整大小。

error_correction: 控制二维码纠错级别。

ERROR_CORRECT_L: 大约7%或者更少的错误会被更正。

ERROR_CORRECT_M: 默认值，大约15%或者更少的错误会被更正。

ERROR_CORRECT_Q: 大约25%或者更少的错误会被更正。

ERROR_CORRECT_H: 大约30%或者更少的错误会被更正。

box_size: 控制二维码中每个格子的像素数，默认为 10。

border: 控制二维码四周留白包含的格子数，默认为4。

'''

检测二维码是否存在

```
from pyzbar import pyzbar    #第三库导入
img = cv2.imread('312.jpg')
texts = pyzbar.decode(img)  #对图片信息进行解码
if not texts: 未找到二维码信息
    pass
    # print('未找到二维码')
else:
    output = str(texts[0].data, encoding='utf-8')
    print(output)
```

二维码的定位及边框绘制

```
def detect(img):
    barcodes = pyzbar.decode(img)
    global img_dst,x,y,w,h
    for barcode in barcodes:
        #提取二维码的边界框的位置
        # 画出图像中条形码的边界框
        (x, y, w, h) = barcode.rect
        # 目标区域y1:y2,x1:x2
        img_dst = img[y - 5:y + h + 5, x - 5:x + w + 5]
    return img_dst, x, y, w, h
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
img_dst, x, y, w, h = detect(gray)
cv2.rectangle(img, (x - 25, y - 25), (x + w + 25, y + h + 25), (0, 0, 255), 8)
# 将二维码区域保存
cv.imwrite('result.jpg', img_dst)
```

对二维码信息进行判别

```
def command_resolve(command):
    last_car_state = 0
    if command.find("Run", 0, len(command)) != -1:
        car_state = 1
    elif command.find("Back", 0, len(command)) != -1:
        car_state = 2
    elif command.find("Left", 0, len(command)) != -1:
        car_state = 3
    elif command.find("Right", 0, len(command)) != -1:
        car_state = 4
    elif command.find("Stop", 0, len(command)) != -1:
        car_state = 0
    else:
        car_state = 5
    # print car_state

    if last_car_state != car_state:
        car_control(car_state)
    last_car_state = car_state
```

#字符串查找指令

```
str.find(str, beg=0, end=len(string))
'''
```

参数

str -- 指定检索的字符串

beg -- 开始索引，默认为0。

end -- 结束索引，默认为字符串的长度。

返回值

如果包含子字符串返回开始的索引值，否则返回-1。

```
'''
```

四、任务要求

- 1、小车能实现二维码识别
- 2、对于二维码位置及二维码信息的不同情况做成合理的运动判断

五、任务实施

- 1、根据下列开发流程，完成二维码识别运动程序



六、任务小结

任务七、人脸识别追踪运动

一、任务描述

二、任务目标

- 1、小车能对人脸进行识别
- 2、根据人脸与小车的位置信息，调整小车的位置

三、相关任务内容

人脸检测

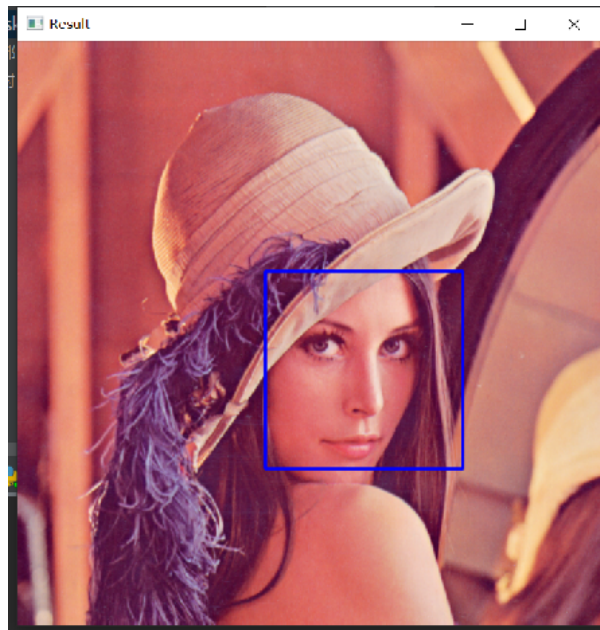
```
import cv2

faceCascade=
cv2.CascadeClassifier("Resources/haarcascade_frontalface_default.xml")
img = cv2.imread('Resources/lena.png')
imgGray = cv2.cvtColor(img,cv2.COLOR_BGR2GRAY)

faces = faceCascade.detectMultiScale(imgGray,1.1,4)

for (x,y,w,h) in faces:
    cv2.rectangle(img,(x,y),(x+w,y+h),(255,0,0),2)

cv2.imshow("Result", img)
cv2.waitKey(0)
```



四、任务要求

- 1、小车能对人脸进行识别
- 2、根据人脸与小车的位置信息，调整小车的位置

五、任务实施

- 1、根据下列开发流程，完成人脸识别追踪运动程序



2、将程序文件传输至树莓派中。

3、通过远程桌面运行该程序文件，按下功能按钮，实现小车运动。

六、任务小结

项目评价

序号	评价项目	自评	师评	得分
1	能够实现小车连接与传输			
2	避障逻辑正确合理			
3	python代码正确有效			